lab6 实验报告

PB24111621 肖瀚

一、实验目的与内容

- 实验目的：在上一次无冒险流水线 CPU 的基础上，添加数据冒险（转发 + 停顿）和控制冒险（分支冲刷）的处理，实现一个完整的五级流水线 RISC-V CPU。
- 实验内容：设计 Forward 前递单元、SegCtrl 分段控制单元（Load-Use 停顿 + 分支冲刷），修改段间寄存器使其支持 stall / flush 双功能，连接 EX 段转发多路数据通路，通过仿真框架验证与上板测试。

二、逻辑设计

总体数据通路

CPU 采用五级流水线结构：IF → ID → EX → MEM → WB。各段间通过流水线寄存器（IF_ID_REG、ID_EX_REG、EX_MEM_REG、MEM_WB_REG）隔离，EX 段集成转发多路选择器、分支判断和冒险控制逻辑。

段间寄存器：stall + flush 双支持

相比 lab5 仅支持 flush，本次需要同时支持 stall（Load-Use 时冻结前两级）和 flush（分支跳转时冲刷前两级，Load-Use 时在 ID/EX 插入气泡）。以 IF/ID 为例：

```
module IF_ID_REG(
    input clk,rst,en,stall,flush,
    input [31:0] if_pc,if_pcadd4,if_inst,
    input commit_instr_if,
    output reg [31:0] id_pc,id_pcadd4,id_inst,
    output reg commit_instr_id,
    output reg [4:0] id_rs1,id_rs2
);
    always @(posedge clk) begin
        if(rst) begin
            // 清零所有寄存器
            id_pc<=0;id_pcadd4<=0;id_inst<=0;
            commit_instr_id<=0;
            id_rs1<=0;id_rs2<=0;
        end else if(en) begin
            if(flush) begin
                // 冲刷：传递PC但清除inst和control → 等效NOP
                id_pc<=if_pc;id_pcadd4<=if_pcadd4;
                id_inst<=0; commit_instr_id<=0;
                id_rs1<=0; id_rs2<=0;
            end else if(!stall) begin
                // 正常传递
                id_pc<=if_pc;id_pcadd4<=if_pcadd4;id_inst<=if_inst;
                commit_instr_id<=commit_instr_if;
                id_rs1<=if_inst[19:15];id_rs2<=if_inst[24:20];
            end
            // stall=1 时：保持当前值，不更新
        end
    end
endmodule
```

优先级为 `rst > flush > !stall > stall`。 `flush` 仅清除控制/数据信号而保留 PC（维持流水线 PC 跟踪）， `stall` 保持所有寄存器不变。

ID_EX_REG 同理，额外支持 flush（分支冲刷/Load-Use 气泡）。EX_MEM 和 MEM_WB 无需 flush（分支在 EX 段已冲刷前方，后方指令正常执行）。

任务一：Forward 解决数据冒险

当 EX 段指令的源寄存器与 MEM 段或 WB 段指令的目的寄存器相同时，直接从后续段前递结果，而非等待写回寄存器堆。

```
module Forward(
    input mem_rf_we,
    input [4:0] mem_wa,
    input [31:0] mem_wd,
    input wb_rf_we,
    input [4:0] wb_wa,
    input [31:0] wb_wd,
    input [4:0] ex_rs1,ex_rs2,
    // 输出：00=reg,01=MEM,10=WB
    output [1:0] fw_a,fw_b,
    output [31:0] fw_data_a,fw_data_b
);
    // MEM 优先级高于 WB（MEM 的数据更新），且排除 x0
    assign fw_a=(mem_rf_we&&mem_wa!=0&&mem_wa==ex_rs1)?2'b01:
                (wb_rf_we&&wb_wa!=0&&wb_wa==ex_rs1)?2'b10:2'b00;
    assign fw_b=(mem_rf_we&&mem_wa!=0&&mem_wa==ex_rs2)?2'b01:
                (wb_rf_we&&wb_wa!=0&&wb_wa==ex_rs2)?2'b10:2'b00;
    assign fw_data_a=(fw_a==2'b01)?mem_wd:wb_wd;
    assign fw_data_b=(fw_b==2'b01)?mem_wd:wb_wd;
endmodule
```

此处实现前递模块生成两位的控制信号 fw_a 与 fw_b，控制一个选择寄存器堆读取数据、alu_res_mem、rf_wd_wb 的三选一 MUX。EX 段 ALU 输入通过 MUX2 四选一多路器选择： `00` =寄存器堆原始值， `01` =MEM 前递， `10` =WB 前递。MEM 段的 `mem_wd` 需特殊处理——对于 JAL/JALR（ `rf_wd_sel == 2'b10` ），前递值应为 `mem_pcadd4` 而非 ALU 的跳转目标地址：

```
wire [31:0] mem_rf_wd_pre = (mem_rf_wd_sel == 2'b10) ? mem_pcadd4 : mem_alu_res;
```

任务二：段间寄存器控制模块

Load-Use 冒险检测与停顿

若 EX 段为 load 指令（ `ex_wd_sel == 2'b01` ），且 ID 段的源寄存器与之相同，则需停顿一个周期并将 ID/EX 段插入气泡：

```
wire load_use = ex_rf_we && (ex_wd_sel == 2'b01) && (ex_wa != 0)
                    && ((id_rs1 == ex_wa) || (id_rs2 == ex_wa));
assign stall_pc      = load_use;    // 冻结PC
assign stall_if_id   = load_use;    // 冻结IF/ID
assign flush_id_ex   = branch || load_use;  // 插入气泡
```

停顿一周期后，load 进入 MEM 段（再下一周期进入 WB），依赖指令进入 EX 段时，load 结果已可从 WB 段前递至 EX。

分支跳转的控制冒险处理

分支在 EX 段计算方向。跳转发生时（ `npc_sel != 00` ），需冲刷 IF/ID 和 ID/EX 两级（等效为 NOP）：

```
wire branch = (npc_sel != 2'b00);
assign flush_if_id = branch;
```

PC 由 NPC_MUX 根据 `npc_sel` 选择下一地址： `00` =PC+4， `01` =PC+offset（分支/JAL）， `10` =PC_j（JALR）。BRANCH 模块使用 **转发后的** `alu_src0_final` 和 `alu_src1_final` 进行比较，确保分支条件判断能获取最新的寄存器值。

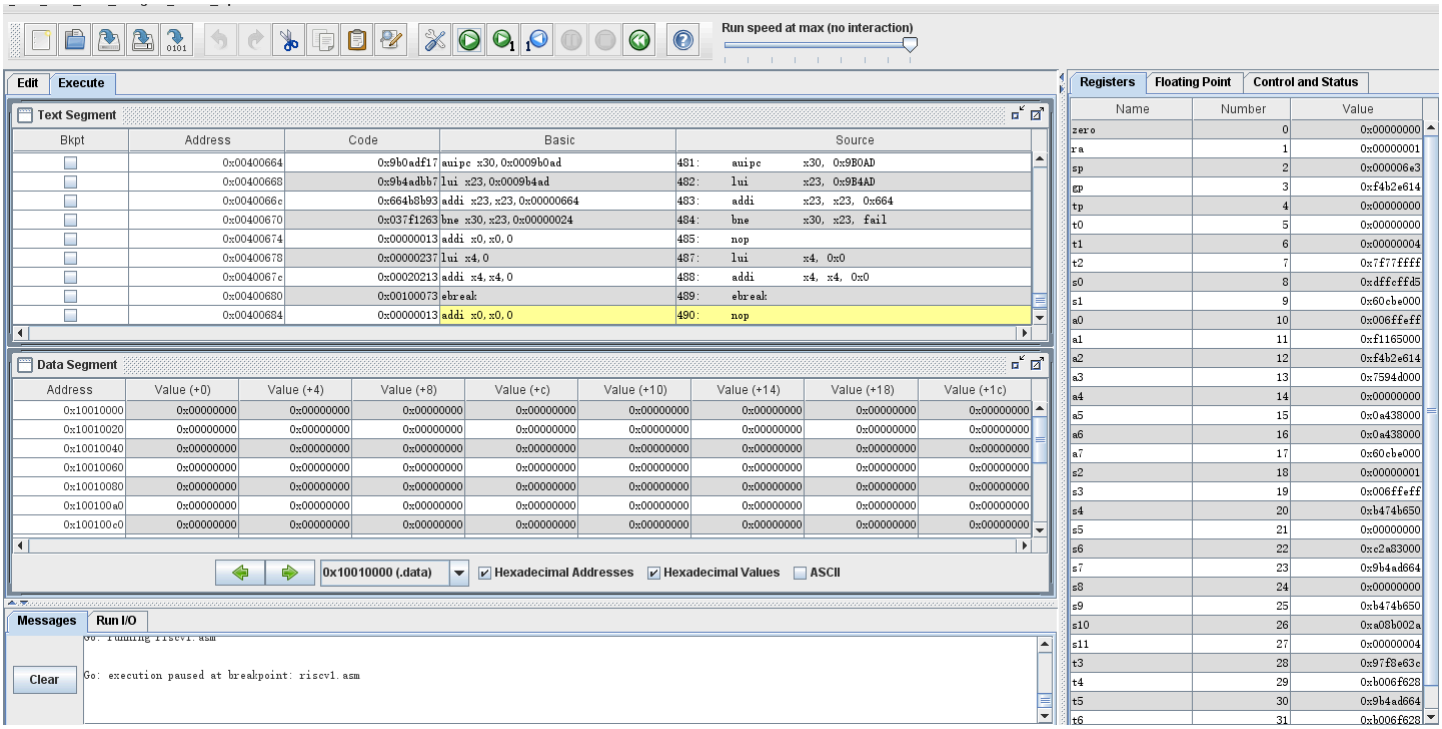
最终的控制模块如下：

```
module SegCtrl(
    // Load-Use：EX是load，ID读相同
    input ex_rf_we,
    input [1:0] ex_wd_sel,
    input [4:0] ex_wa,
    input [4:0] id_rs1,id_rs2,
    // 分支
    input [1:0] npc_sel,
    // 输出
    output stall_pc,stall_if_id,
    output flush_if_id,flush_id_ex
);
    // Load-Use：ex是load（wd_sel=1），且id_rs1/2==ex_wa≠0
    wire load_use=ex_rf_we&&(ex_wd_sel==2'b01)&&(ex_wa!=0)&&
                  ((id_rs1==ex_wa)||(id_rs2==ex_wa));
    // 分支跳转：npc_sel≠0
    wire branch= (npc_sel!=2'b00);

    assign stall_pc=load_use;
    assign stall_if_id=load_use;
    assign flush_if_id=branch;
    assign flush_id_ex=branch||load_use;
endmodule
```

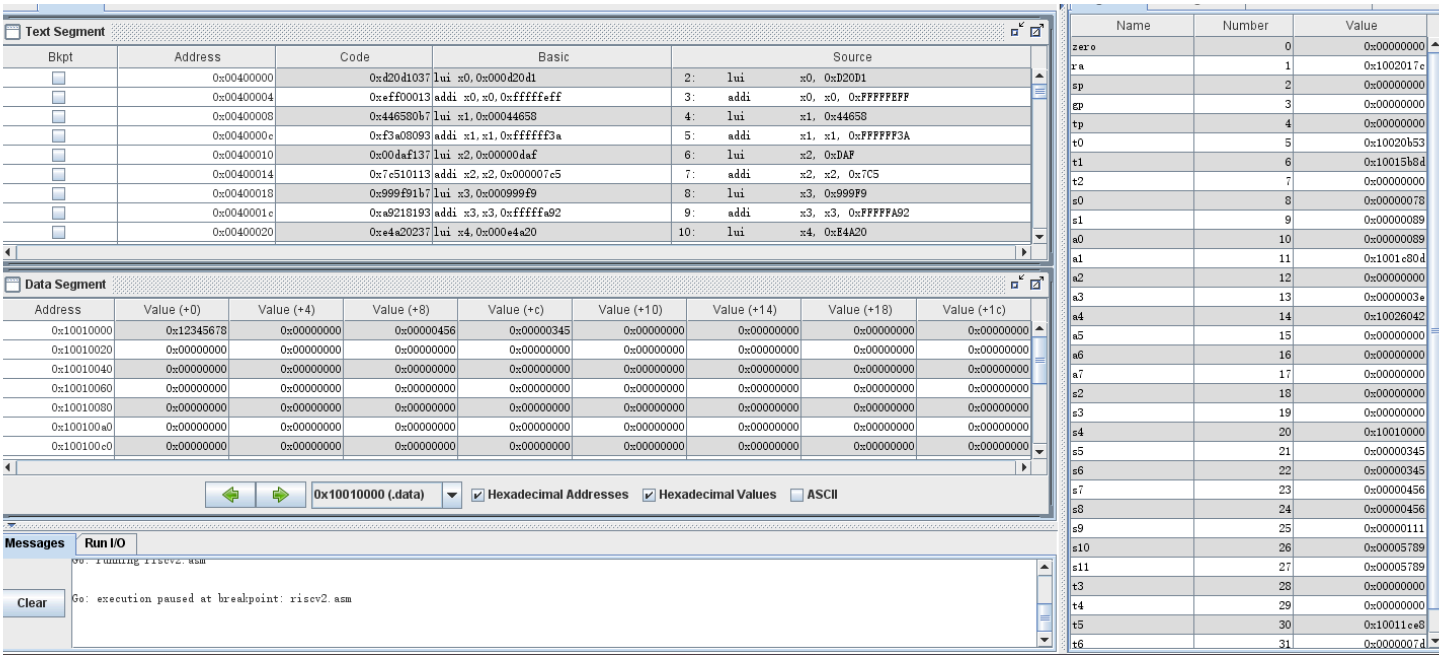
三、仿真验证

对第一个考虑前递的测试仿真，结果如下：

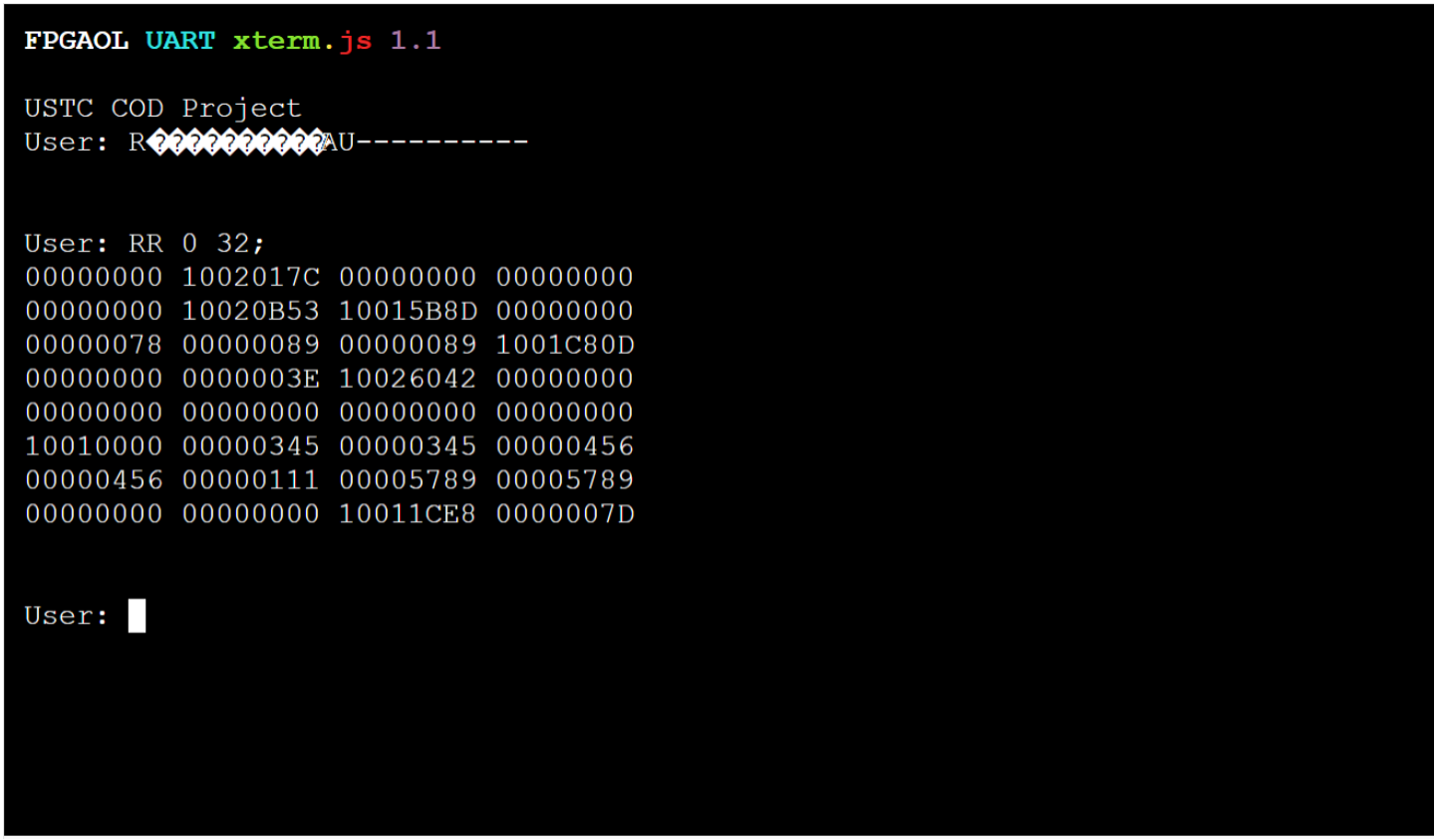



```
Dumping waveform to waveform/waveform.vcd.
Hit good trap.
Simulation finished.
Time:                           844
Instruction count:              417
PC:                             0x00400680
Instruction:                    0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x00000001 [ 2] = 0x000006E3 [ 3] = 0xF4B2E614
[ 4] = 0x00000000 [ 5] = 0x00000000 [ 6] = 0x00000004 [ 7] = 0x7F77FFFF
[ 8] = 0xDFFCFFD5 [ 9] = 0x60CBE000 [10] = 0x006FFEFF [11] = 0xF1165000
[12] = 0xF4B2E614 [13] = 0x7594D000 [14] = 0x00000000 [15] = 0x0A438000
[16] = 0x0A438000 [17] = 0x60CBE000 [18] = 0x00000001 [19] = 0x006FFEFF
[20] = 0xB474B650 [21] = 0x00000000 [22] = 0xC2A83000 [23] = 0x9B4AD664
[24] = 0x00000000 [25] = 0xB474B650 [26] = 0xA08B002A [27] = 0x00000004
[28] = 0x97F8E63C [29] = 0xB006F628 [30] = 0x9B4AD664 [31] = 0xB006F628
qi@ISLA:~/COD25-Simulation-Framework$
```

对于第二个完整 CPU 的测试仿真，结果如下：




```
qi@ISLA:~/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Difftest level: FULL.
Dumping waveform to waveform/waveform.vcd.
Hit good trap.
Simulation finished.
Time:                           1226
Instruction count:              492
PC:                             0x004007E8
Instruction:                    0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x1002017C [ 2] = 0x00000000 [ 3] = 0x00000000
[ 4] = 0x00000000 [ 5] = 0x10020B53 [ 6] = 0x10015B8D [ 7] = 0x00000000
[ 8] = 0x00000078 [ 9] = 0x00000089 [10] = 0x00000089 [11] = 0x1001C80D
[12] = 0x00000000 [13] = 0x0000003E [14] = 0x10026042 [15] = 0x00000000
[16] = 0x00000000 [17] = 0x00000000 [18] = 0x00000000 [19] = 0x00000000
[20] = 0x10010000 [21] = 0x00000345 [22] = 0x00000345 [23] = 0x00000456
[24] = 0x00000456 [25] = 0x00000111 [26] = 0x00005789 [27] = 0x00005789
[28] = 0x00000000 [29] = 0x00000000 [30] = 0x10011CE8 [31] = 0x0000007D
qi@ISLA:~/COD25-Simulation-Framework$
```



四、总结

本次实验在 lab5 无冒险流水线 CPU 的基础上，实现了完整的冒险处理机制：Forward 前递单元解决 EX→EX 和 MEM→EX 的数据冒险，SegCtrl 分段控制器处理 Load-Use 停顿和分支冲刷。本次实验加深了对流水线冒险产生原因和解决方法的理解，特别是前递优先级（MEM>WB）、Load-Use 必须停顿一拍的原理。